

Milestone 2: Hazard Detection and Resolution

Forwarding is done in `gen_forward` and the load-use hazard is detected in `detect_hazard`, both in `stage_helpers.h`. The stall bubble and the pipeline flush are applied at the end of each cycle in `cycle_pipeline` in `pipeline.c`.

EX Hazard

An EX hazard occurs when the instruction entering EX needs a register that the previous instruction, now in the MEM stage, is going to write. The result already exists in the EX/MEM pipeline register. The condition in `gen_forward` is checked once per source register:

```
bool ex_rs1 = idex->use_rs1 && idex->rs1 != 0 &&
              exmem->reg_write && exmem->rd == idex->rs1;
bool ex_rs2 = idex->use_rs2 && idex->rs2 != 0 &&
              exmem->reg_write && exmem->rd == idex->rs2;
```

This reads: the instruction entering EX actually uses this source register, the register is not x0 since x0 is hard-wired to zero, the instruction in MEM writes a destination register, and that destination equals our source. To resolve the hazard we overwrite the operand value that decode read from the register file:

```
idex->rs1_val = exmem->alu_result; // same for rs2_val
fwd_exex_counter++;
```

The EX stage then computes with the forwarded value, so a dependent instruction directly after its producer runs without stalling.

MEM Hazard

A MEM hazard occurs when the needed register is written by the instruction two ahead, now in the WB stage. Its result sits in the MEM/WB pipeline register. The condition is:

```
bool mem_rs1 = !ex_rs1 && idex->use_rs1 && idex->rs1 != 0 &&
               memwb->reg_write && memwb->rd == idex->rs1;
bool mem_rs2 = !ex_rs2 && idex->use_rs2 && idex->rs2 != 0 &&
               memwb->reg_write && memwb->rd == idex->rs2;
```

It is the same match check, but against the MEM/WB register and with one extra term: `!ex_rs1` or `!ex_rs2`. If both older instructions write the same register, the EX hazard must win because the instruction in MEM is more recent, so the MEM hazard only fires for an operand the EX hazard did not cover. To resolve it we forward the value WB is writing back this cycle. For a load this is the data read from memory, otherwise the ALU result:

```
uint32_t wb_val = memwb->mem_to_reg ? memwb->mem_result
                                     : memwb->alu_result;
idex->rs1_val = wb_val; // same for rs2_val
fwd_exmem_counter++;
```

Load-Use Hazard

Forwarding cannot help when a load is immediately followed by an instruction that uses the loaded register: when that instruction wants to enter EX, the load is still in MEM and the data has not been read yet. `detect_hazard` compares the load in the ID/EX register against the instruction in the IF/ID register:

```
if (!idex->mem_read || idex->rd == 0)
```

```

    return;

    gen_reg_fields(ifid->instr, &rs1, &rs2, &rd, &use_rs1, &use_rs2);

    if ((use_rs1 && rs1 == idex->rd) || (use_rs2 && rs2 == idex->rd))
    {
        pwires_p->stall    = true;
        pwires_p->pc_src0 = regfile_p->PC;
        stall_counter++;
    }

```

mem_read is only true for loads, so the condition means: a load is in EX, it writes a real register, and the instruction behind it in ID reads that register as rs1 or rs2. The helper gen_reg_fields returns the source register numbers of the ID instruction plus flags saying whether each one is really used. The flags matter because the rs1 and rs2 bit positions overlap immediate bits in other instruction formats, and without them an immediate could look like a matching register and cause a false stall. On a hazard we raise the stall wire and rewrite pc_src0 to the current PC, so fetch repeats the same instruction next cycle. The dependent instruction is delayed one cycle, after which the load is in WB and normal MEM-hazard forwarding delivers the loaded value.

Pipeline Flush

Branches and jal are resolved in the MEM stage. The EX stage computes branch_taken and branch_target and they travel with the instruction through the pipeline registers. At the end of every cycle cycle_pipeline checks the value about to enter MEM/WB:

```

    if (pregs_p->memwb_preg.inp.branch_taken)
    {
        pwires_p->pcsrc    = true;
        pwires_p->pc_src1 = pregs_p->memwb_preg.inp.branch_target;
        branch_counter++;
        ifid_f.instr.bits = 0x00000013;    // and idex_f, exmem_f
        ...
    }

```

When the branch is taken, the three instructions behind it were fetched from the wrong path and must have no effect. Each of the IF/ID, ID/EX and EX/MEM registers is replaced with a NOP: the instruction bits become 0x00000013, which is addi x0, x0, 0, all control signals are cleared to zero, and only the instruction address is kept so the trace still shows where the killed slot came from. At the same time pcsrc and pc_src1 redirect fetch to the branch target.

Stall and Bubble Insertion

When detect_hazard raises the stall wire, cycle_pipeline inserts the bubble before the pipeline registers are latched:

```

    pregs_p->ifid_preg.inp = pregs_p->ifid_preg.out;    // hold in id
    pregs_p->idex_preg.inp.reg_write = false;          // squash the copy
    pregs_p->idex_preg.inp.mem_read  = false;
    pregs_p->idex_preg.inp.mem_write = false;
    pregs_p->idex_preg.inp.branch   = false;
    pregs_p->idex_preg.inp.jump     = false;

```

Two things happen. First, the IF/ID register is reloaded with its own current value, so the dependent instruction stays in ID and decodes again next cycle. Since the PC was rewritten, fetch also repeats, so nothing is lost. Second, the copy of the instruction that was already decoded this cycle becomes the

bubble: it keeps its instruction bits, but every control signal that could cause an effect is cleared, so as it moves through EX, MEM and WB it cannot write a register, touch memory, or resolve as a branch. The held copy follows one cycle behind and gets the correct value through the MEM-hazard forward.

Other Changes

Pipeline register fields in pipeline.h. We added `rs1`, `rs2`, `use_rs1` and `use_rs2` to `idex_reg_t` because the forwarding and hazard units need the source registers of an instruction without re-decoding it. We added `branch_taken` and `branch_target` to the EX/MEM and MEM/WB registers so the branch decision made in EX can travel with the instruction and be acted on in MEM. A `stall` flag was added to the pipeline wires to carry the hazard decision to the end of the cycle.

Branch handling moved out of execute. In milestone 1 the PC redirect happened as soon as the branch executed. Now `stage_execute` only computes the outcome, using the forwarded operand values so a branch close behind its producer compares the right numbers, and the redirect plus flush happen at the MEM/WB boundary. This was required because milestone 2 defines the branch as resolved in the MEM stage.

Cycle ordering in `cycle_pipeline`. The order is `fetch`, `detect_hazard`, `decode`, `gen_forward`, `execute`, `mem`, `writeback`, then the stall and flush handling. `detect_hazard` must run after `fetch` so it can rewrite the PC for the next cycle, and `gen_forward` must run between `decode` and `execute` so EX sees the forwarded values.

Printing the empty startup slots. At reset the pipeline registers hold `0x00000000`, which is not a valid opcode, and `parse_instruction` exits on it. A small `print_instruction` wrapper prints a blank line for zero bits and calls `decode_instruction` otherwise, matching the reference traces without crashing.

ALU additions. The test programs use `mulh`, `div` and `rem`, which our milestone 1 ALU decoded as `sll`, `xor` and `or` because it ignored `funct7`. `gen_alu_control` now checks `funct7 == 0x01` and `execute_alu` implements all three. The divide cases guard against a zero divisor because a wrong-path instruction that later gets flushed still passes through the ALU and must not crash the simulator.